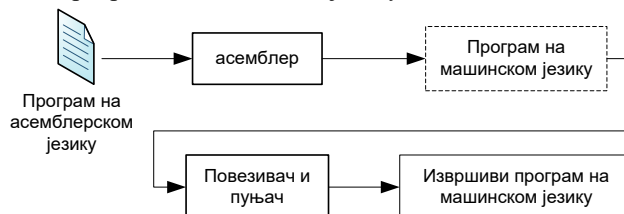


АСЕМБЛЕР

Асемблер је програм који преводи изворни програм написан на асемблерском језику у програм на машинском језику и извршиви програм на машинском језику.

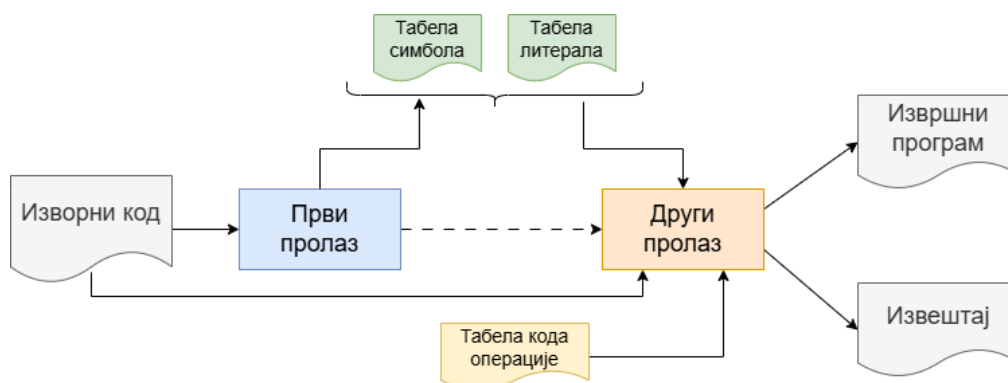


Слика 1: Процес преводијења са асемблерског језика

Асемблер пролази линију по линију кода и одређује да ли се на линији налази:

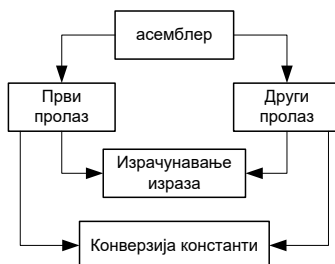
- **Коментар** – текст у програму који служи за објашњење и не утиче на извршавање кода.
- **Директива** – наредба за организацију кода (дефинисање секција, резервисање меморије).
- **Симбол** – ентитет у програму (нпр. променљива, лабела или адреса).
- **Литерал** – директна вредност (константа) написана у коду (нпр. број или текст).

Због потребе за познавањем вредности симболичких израза у инструкцијама (нпр. лабеле за скокове, имена променљивих) процес асемблирања се реализује у **два пролаза**. Први пролаз асемблера **проналази симболе и рачуна адресе**, док други генерише **машински код**.



Слика 2: Први и други пролаз асемблера

Оба пролаза користе потпрограме за израчунавање израза и конверзију константи (Слика 3), који су заједно са табелама објашњени у даљем тексту, али прилагођени примеру малог асемблера који се користи на овом курсу, а који је близак данашњем MIPS асемблеру.



Слика 3: Однос између модула асемблера

Први пролаз

У првом пролазу, асемблер пролази кроз цео код, линију по линију, прати бројач локација и рачуна где ће се свака инструкција или податак сместити у меморију. Основни задаци првог пролаза асемблера су:

- Препознавање свих **симбола** (лабела) и **литерала** (константи) који се у програму користе.
- Креирање **табеле симбола** и **табеле литерала**.
- Препознавање директива (нпр. `.data`, `.org`, `.word`) и **управљање бројачем мем. локација**.

На крају првог пролаза асемблер зна где се налази **свака лабела и симбол**, али још нема комплетног машинског кода.

У току првог пролаза асемблер мора доделити меморију за:

- **сваку инструкцију** која се генерише из симболичке инструкције (нпр. `sub $7, $8, 1`),
- **сваки литерал** који се генерише псеудооперацијом $\alpha: .word \beta$ (α – симбол, β – литерал),
- **сваки меморијски блок** резервисан исказом `.space P`,
- **сваку секцију** која се дефинише псеудооперацијом `.org S`.

Када асемблер наиђе на машинску инструкцију у улазној датотеци, он мора одредити колико меморијских локација треба да заузме за ту инструкцију. Код неких циљних платформи (најчешће MISC типа) величина инструкције није увек иста, те се она може одредити тек након утврђивања типа инструкције; док је код неких других платформи (RISC типа) величина инструкције константна. У нашем примеру ради се о платформи са **константном величином инструкције**, која износи **32 бита**.

Међутим, податак о величини инструкције није довољан да би асемблер знао колико меморијских локација јој мора доделити. Потребно је знати и организацију меморије у циљној платформи.

Основне карактеристике меморијског подсистема су:

- **адресни простор** – са колико бита је дефинисана меморијска адреса,
- **величина** – максимална величина меморије, која у стварности може бити мања,
- **ширина** – количина података која се налази на једној меморијској адреси, тј. локацији.

Додатни податак који је потребан је управо **ширина меморије**. У нашем примеру ширина је **8 бита**, тако да асемблер једној инструкцији мора доделити 4 ($32/8=4$) узастопне меморијске локације, тј. **4 бајта**.

У овом примеру величина података над којима се обављају операције је исто 32 бита. Зато се каже да је платформа 32-битна, тј. да је једна рачунарска реч 32 бита. Тако да се и за сваки **литерал** заузимају **4 меморијске локације**.

Унутар додељеног меморијског простора не сме да се појави ни један (прескочен) бајт, те је потребно управљати **бројачем меморијских локација**, који указује на недодељен бајт у меморији.

Након обраде сваког исказа, бројач се увећава за одређену вредност у зависности од типа:

Тип	Пример	Промена бројача мем. локација
Инструкција	<code>sub \$4, \$7, \$1</code>	+4
Литерал	<code>.word 9</code>	+4
Блок меморије	<code>.space P</code>	+(P по модулу 4)
Симбол	Завршава се са „:“	Не мења се
Секција	<code>.data</code> или <code>.text</code>	Бројач се поставља на 0 за сваку секцију

За сваки пронађени **симбол** проверава се да ли већ постоји исти симбол у *табели симбола*. У случају да не постоји, симбол се додаје у табелу, а у случају да постоји, пријављује се грешка (јер симбол не сме бити редефинисан).

НАПОМЕНА: Специјалне врсте симбола, нпр. **глобални** (енгл. global), тј. јавни смештају се у посебне табеле глобалних (јавних) симбола.

За сваки пронађени **литерал (константу)** формира се нова врста у *табели литерала* у коју се он додаје. Вредност литерала је константа и она се израчунава помоћу потпрограма за *конверзију константи*. Приликом преузимања параметара операција и псеудооперација користи се потпрограм за *израчунавање израза*.

Следи пример полазног кода у асемблерском језику:

```
.globl main
.data
    value: .word 9
.text

main:
    la    $t1, value      # učitavanje adrese
    lw    $t2, 0($t1)     # učitavanje vrednosti faktoriјela sa adrese u $t1
    add   $t3, $t2, $0     # postavljanje rezultata na value
    addi  $t1, $0, 1
    sub   $t2, $t2, $t1    # umanјivanje broјaca
    blez  $t2, exit
    nop

loop:
    mult  $t3, $t2
    MFLO  $t3
    sub   $t2, $t2, $t1    # umanјivanje broјaca
    blez  $t2, exit
    nop
    b     loop

exit: nop
```

Задатак: Анализирати програм линију по линију кода и пронаћи све директиве, симболе и литерале, а затим избројати меморијске локације за сваку симболичку инструкцију.

Напомена: Инструкција *la* је пресудоинструкција која се састоји из 2 инструкције *lui + ori*.

Након првог пролаза асемблера добијају се следеће табеле литерала и симбола:

Табела симбола				Табела литерала	
.data – 0	меморијска	симбол		вредност	меморијска
.text – 1	адреса				адреса
0	0x00000000 (0)	value		0x000000009	0x00000000
1	0x00000000 (0)	main			
1	0x00000020 (32)	loop			
1	0x00000038 (56)	exit			

ПИТАЊЕ: Која би била адреса симбола *main*, да се пре *main*-а налази и линија: .space 7

Други пролаз

У другом пролазу асемблер, користећи резултате првог пролаза, врши превођење симболичког кода у **машински код**. За разлику од првог пролаза, у коме се одређују адресе симбола и формирају табеле, у другом пролазу се генеришу коначне машинске инструкције. У те сврхе, неопходно је одређивање сваког поља у инструкцији. Вредности кодова операције се одређују на основу вредности које се налазе у **табели кода операције** (Табела 3).

Основни задаци другог пролаза су:

- одређивање машинског кода за сваку инструкцију,
- замена симбола њиховим стварним адресама,
- израчунавање вредности операнда,
- попуњавање поља инструкције (opcode, регистри, непосредне вредности и др.).

Приликом обраде сваке линије кода, асемблер:

1. препознаје инструкцију и њен формат,
2. одређује код операције (opcode),
3. обрађује операнде (регистре, константе, адресе),
4. користи табелу симбола за замену лабела одговарајућим адресама,
5. формира бинарни запис инструкције.

Напомена: У оквиру ове вежбе се реализује само први пролаз асемблера.

Табеле у асемблеру (улазни подаци за други пролаз асемблера)

Табела 1: Табела симбола

Симбол	Вредност (меморијска локација)
N	0x00000018

Табела 2: Табела литерала

Константа (бројна вредност)	Меморијска локација
0x40	0x00001048

Табела 3: Табела кода операције

Симбол	Тип операције	Формат ¹	Нумерички код	Функција	Број параметара
add	машинска	R	0x0	0x20	3

Реализација модула и функција од интереса у библиотеци *assemblerLib*

Први пролаз асемблера

Први пролаз асемблера је, ради једноставности, подељен у два дела, односно учитавање линија кода у листу и замена псеудо инструкција је извучена у посебан модул *sourceLoader*. Модул је већ реализован у библиотеци и његова једина функција обавља наведене операције, односно попуњава листу (глобалну променљиву) *sourceList*:

```
bool loadSourceRepPseudo(std::string input_file)
```

Главни део првог пролаза остављен је за модул *passI* и његова једина функција је **нереализована**:

```
bool passI()
```

Табела симбола

Табела симбола је реализована као мапа са симболом (кључ) и меморијском локацијом (придружени податак), у модулу *assemblerLib*:

```
typedef std::map<std::string, Symbol> SymbolTable; // мапа
void pushSymbol(std::string symbol, long location, Section section); // додавање елемента
bool readSymbol(std::string symbol, long &val); // преузимање и измена ел.
bool symbolExists(std::string symbol); // провера постојања ел.
void pushGlobalSymbol(std::string symbol, long location); // додавање глобалног симбола
bool globalSymbolExists(std::string symbol); // провера постојања глобалног сим.
void pushExternSymbol(std::string symbol, long location); // додавање спољашњег симбола
bool externSymbolExists(std::string symbol); // провера постојања спољашњег сим.
```

Табела литерала

Табела литерала је реализована као мапа са вредношћу литерала (кључ) и меморијском локацијом (придружени податак), у модулу *assemblerLib*:

```
typedef std::map<long, Literal> LiteralTable; // мапа
void pushLiteral(long location, long value); // додавање ел.
bool readLiteral(long location, long &value); // читање и измена ел.
```

Листе линија кода и бројева линија

Свака линија изворног кода представљена је структуром која садржи саму линију кода и редни број те линије у улазној датотеци:

```
struct SourceLine {
    long lineNumber;
    std::string sourceLine;
};
```

Листа линија кода и бројева линија (структура *SourceLine*), као и функција добављања показивача на глобалну листу наведене су испод, а реализоване су у модулу *assemblerLib*:

```
typedef std::list<SourceLine> SourceList;    // листа
SourceList& getSourceList(); // добављање референце на листу са изворним
                                   // кодом (глобална променљива у библиотеци assemblerLib
```

Листа параметара

Листа параметара псеудооперације или инструкције је у библиотеци реализована као стек.

```
typedef std::stack<std::string> ParamStack;    // стек
```

Помоћне функције при првом пролазу

Конверзија константи:

```
long constConv(std::string constant, bool &isNum);
```

Ишчитавање дела линије кода са извршивом инструкцијом:

```
bool getExecutable(std::string &executable, std::string line, long lineNumber, bool
    addErrorToList=false);
```

Ажурирање тренутне секције section и постављање бројача меморијских локација location на последњу адресу секције sectionId:

```
bool changeSectionAndLocation(std::string sectionId, Section &section, long &location);
```

Ишчитавање директиве из линије кода:

```
Directive getDirective(std::string line, long lineNumber);
```

Ишчитавање симбола из линије кода:

```
bool getSymbol(std::string &symbol, std::string line);
```

Ишчитавање свих параметара из линије кода:

```
bool getParams(PARAM_STACK &params, std::string line, long lineNumber, bool
    chopDollar=false, bool unwind=false, long location=0);
```

Провера једнакости актуелног броја параметара и очекиваног броја параметара:

```
bool checkEnoughParams(long lineNumber, std::string line, PARAM_STACK params,
    int nExpected);
```

Пријављивање грешке додавањем поруке у листу грешака:

```
void addError(long lineNumber, std::string line, std::string errorText);
```

Провера постојања пријављених грешака и испис порука о грешкама, уколико постоје:

```
bool errorsFound();
```

Детаљнији опис ових и других функција можете прочитати у *html* документацији библиотеке *assemblerLib* – *doc/html/index.html*.

ЗАДАТАК

1. Направити текстуалну датотеку *ИмеПрезимеБројИндекса* и одговорити на наредна питања:
 - а. Шта је улаз, а шта излаз из првог пролаза асемблера?
 - б. Шта је улаз, а шта излаз из другог пролаза асемблера?
 - с. Шта подразумева конверзија константи?
2. Реализовати употребом програмског језика C/C++ и већ постојеће библиотеке асемблера *AssemblerLib*, први пролаз асемблера у датотеци *pass1.cpp*. Искористити постојећи пројекат *small_assembler*. Потребно је попунити табелу симбола, док је табела литерала већ попуњена. За њено попуњавање потребно је водити рачуна о **бројачу меморијских локација**. У пројекат је већ укључена библиотека *AssemblerLib.lib* и дефинисане су путање до додатних датотека (заглавља). Са циљем једноставније реализације користити постојеће функције за конверзију константи, издвајање директива, издвајање симбола, проверу да ли је линија извршна, проверу броја параметара, као и све функције за приступ табелама које су потребне у првом пролазу.

ДОДАТНИ ЗАДАТАК

Реализовати две нове псеудо-инструкције, додавањем њихових дефиниција у низ псеудо-инструкција *PSEUDO_INSTRUCTIONS* у датотеци *pseudoInstructionList.h*:

Симбол	Параметар 1	Параметар 2	Параметар 3
bgt ²	Регистар	Регистар	Лабела/одстојање (вредност)
blt ³	регистар	регистар	Лабела/одстојање (вредност)

Добар пример прављења псеудо-инструкције би могла бити псеудо-инструкција *nori* у постојећој листи псеудо-инструкција. Листа асемблерских инструкција дата је у додатку MIPS-instructions.pdf.

² енг. branch on greater than

³ енг. branch on less than